

LABORATORIO 14

Qué cambió desde ayer

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo.

Introducción

1 · El tema

Un proceso nocturno lee tu tabla todas las noches y la copia a otro sistema. Anoche la tabla tenía treinta mil filas y hoy tiene treinta mil una. La pregunta del laboratorio es si ese proceso tiene que volver a leer las treinta mil para encontrar la que cambió.

La respuesta es que no. La tabla sabe qué trajo cada página, y se le puede pedir solo el movimiento.

2 · El problema, en el almacén

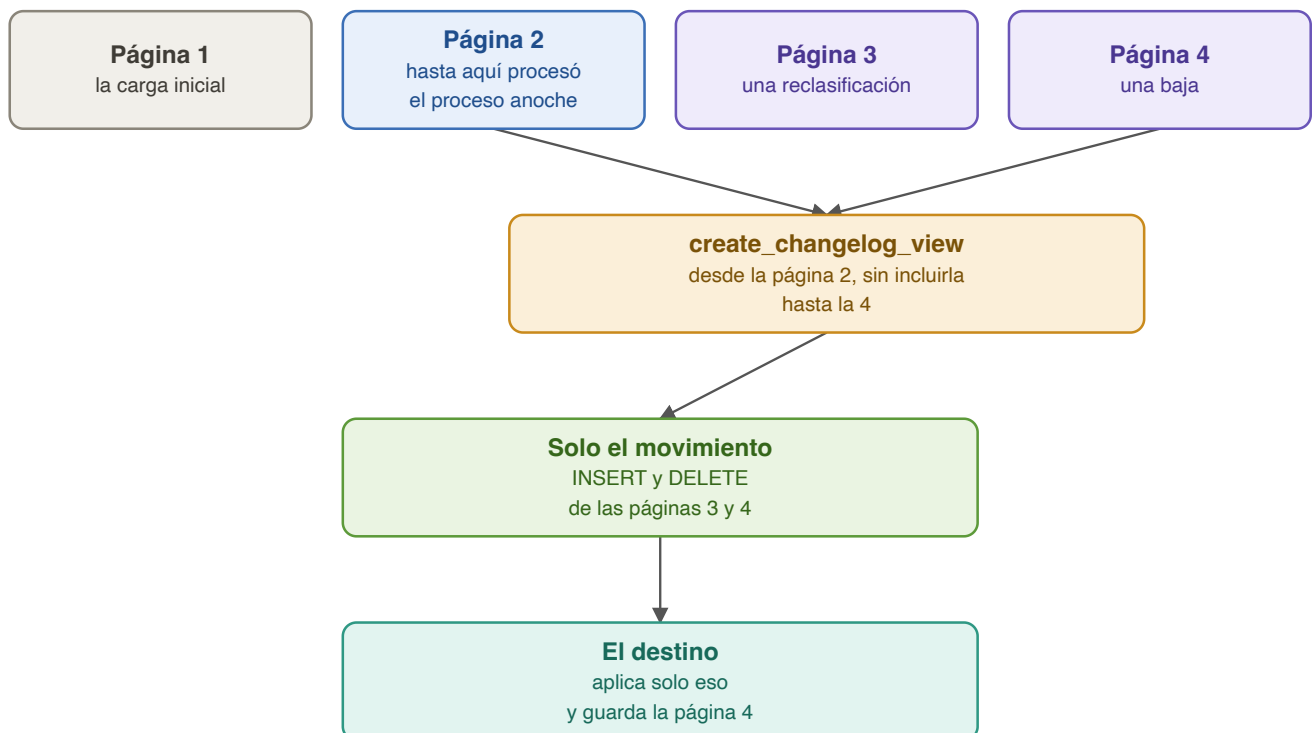
El contador del almacén pasa cada noche a copiar la libreta a su propio sistema. Si cada noche copia la libreta entera, cuando la libreta crece se pasa la noche copiando, y la mayor parte de lo que copia no cambió. Lo que necesita es la lista de lo que se anotó, se corrigió y se tachó desde la última vez que pasó.

3 · El problema en términos técnicos

Sin ayuda de la tabla, un proceso incremental tiene tres caminos malos. Releer todo y comparar, que cuesta lo mismo que la tabla entera. Pedirle al origen una columna con la fecha de modificación, que alguien tiene que mantener. O poner disparadores y una tabla de bitácora al lado, que hay que cuidar y que se desincroniza.

Una consulta normal devuelve la foto de la tabla, cómo está. Lo que el proceso necesita es otra cosa, qué filas entraron, cuáles salieron y cuáles cambiaron entre dos momentos.

4 · El diagrama



Página 1 (gris). La carga inicial de la tabla.

Página 2 (azul). La última página que el proceso nocturno ya copió. Es su marca. Todo lo anterior ya está en el destino.

Página 3 y Página 4 (morado). Lo que pasó después, una reclasificación y una baja. Es lo que falta copiar.

create_changelog_view (amarillo). Es el procedimiento de Iceberg que arma una vista con los cambios entre dos páginas. Se le da la página desde la cual mirar, sin incluirla, y la página hasta donde mirar.

Solo el movimiento (verde). La vista trae una fila por cada cambio, marcada como **INSERT** o **DELETE**, con la página en que ocurrió.

El destino (verde agua). El proceso aplica solo esos cambios en el otro sistema y guarda la página 4 como su nueva marca para la noche siguiente.

5 · La solución

En el almacén, cada página de la libreta tiene anotado qué renglones se agregaron y cuáles se tacharon. El contador ya no copia la libreta entera. Anota en su cuaderno hasta qué página copió, y la noche siguiente pide solo lo que se escribió desde esa página.

En Iceberg eso es la **lectura incremental**. `create_changelog_view` construye una vista con el movimiento entre dos páginas, a partir de lo que la tabla ya tiene anotado. No hace falta ninguna columna de auditoría, ningún disparador ni ninguna bitácora al lado. El costo depende de lo que cambió, no del tamaño de la tabla.

6 · Los pasos

- **Paso 0.** Creas `contribuyentes_lab14` y cargas cuatro contribuyentes, que es la carga de anoche.
- **Paso 1.** Pasa el día. Entran dos contribuyentes nuevos, uno se reclasifica y otro se da de baja, y miras las páginas.
- **Paso 2.** Pides la vista de cambios de toda la historia y la lees.
- **Paso 3.** Pides solo los cambios entre la segunda página y la última.
- **Paso 4.** Ves cómo usa esto un proceso nocturno de verdad.
- **Cierre.** La foto contra el movimiento.

Si quieres repetir el laboratorio desde cero, borra la tabla con el comando de abajo, desde la carpeta del repositorio. El paso 0 igual empieza con un `DROP TABLE IF EXISTS`.

```
bash
bin/reiniciar-lab.sh 14
```

Paso 0 · La tabla de hoy

Celda 0.1 · Entrar a tu espacio

La pregunta. ¿En qué espacio de trabajo vas a crear la tabla?

Por qué ahora. Las celdas que siguen nombran la tabla sin el espacio adelante, y eso funciona solo si estás parado en el tuyo.

En el almacén. Es pararse frente a tu propio estante antes de empezar.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Todo lo que va después de dos guiones lo ignora el motor.
- `USE mi_espacio` te deja parado en tu espacio de trabajo, `mi_espacio`.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:24:49 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La sentencia funcionó. `Listo. La sentencia se ejecutó.` es lo que escribe el cuaderno cuando una sentencia no devuelve filas.

Las tres primeras líneas no son errores. Salen solo en la primera celda que usa Spark, cuando arranca.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:24:49` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.

Celda 0.2 · Partir de cero

La pregunta. ¿Hay una tabla de una corrida anterior que estorbe?

Por qué ahora. La historia de la tabla tiene que empezar con la carga de este paso, o los números de página no calzan con lo que sigue.

En el almacén. Es sacar del estante la libreta vieja.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla.
- `IF EXISTS` hace que no reclame si la tabla no existe.
- `contribuyentes_lab14` es el nombre de la tabla.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS contribuyentes_lab14
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se borró, o no existía. En los dos casos sale lo mismo.

Celda 0.3 · Crear la tabla

La pregunta. ¿Cómo es la tabla del día?

Por qué ahora. Un padrón chico, de tres columnas, para poder seguir cada cambio con la vista.

En el almacén. Es abrir una libreta nueva con tres casillas por renglón.

La sentencia, parte por parte.

- `CREATE TABLE contribuyentes_lab14` crea la tabla.
- `rut STRING` es el RUT del contribuyente, como texto.
- `razon_social STRING` es el nombre de la empresa.
- `segmento STRING` es su tamaño, MICRO, PEQUENA, MEDIANA o GRANDE.
- `USING iceberg` hace que sea una tabla Iceberg.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE contribuyentes_lab14 (
    rut          STRING,
    razon_social STRING,
    segmento     STRING
) USING iceberg
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La tabla existe, vacía y sin páginas.

Celda 0.4 · La carga de anoche

La pregunta. ¿Con qué parte la historia de la tabla?

Por qué ahora. Esta carga deja la primera página, el punto de partida de todo lo que viene.

En el almacén. Es escribir los primeros cuatro renglones de la libreta.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab14` agrega filas a la tabla.
- `VALUES` trae las filas escritas a mano, una por paréntesis, separadas por comas. Cada una tiene RUT, razón social y segmento, en el orden de las columnas.

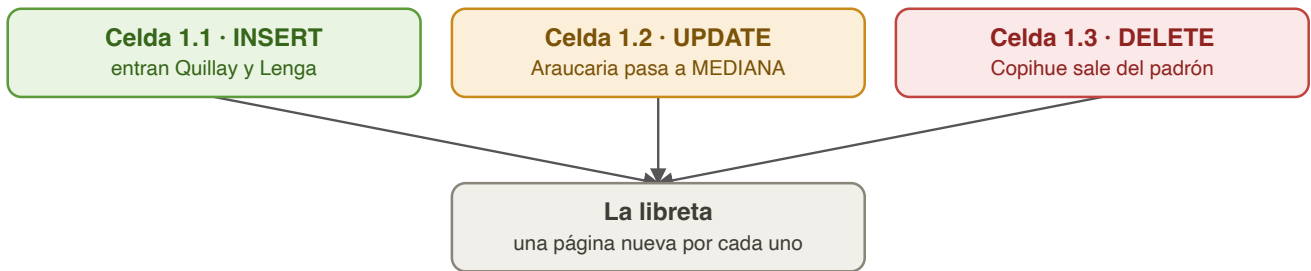
```
sql
%%sql
-- Celda 0.4
INSERT INTO contribuyentes_lab14 VALUES
  ('77746521-K', 'Pehuen Logistica EIRL', 'MICRO'),
  ('77884562-8', 'Araucaria Ferreteria SpA', 'PEQUENA'),
  ('78800840-6', 'Copihue Maquinarias EIRL', 'MEDIANA'),
  ('76171162-8', 'Huemul Alimentos EIRL', 'GRANDE')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entraron los cuatro contribuyentes, Pehuén, Araucaria, Copihue y Huemul.

Paso 1 · Un día de movimientos



Celda 1.1 · INSERT (verde). Entran dos contribuyentes nuevos.

Celda 1.2 · UPDATE (amarillo). A Araucaria la reclasifican de PEQUENA a MEDIANA.

Celda 1.3 · DELETE (rojo). A Copihue la dan de baja del padrón.

La libreta (gris). Cada uno de los tres movimientos cuelga su propia página.

Celda 1.1 · Dos contribuyentes nuevos

La pregunta. ¿Qué deja en la libreta una carga de filas nuevas?

Por qué ahora. Es el primer movimiento del día.

En el almacén. Es agregar dos renglones al final de la libreta.

La sentencia, parte por parte.

- `INSERT INTO contribuyentes_lab14` agrega filas.
- `VALUES` trae las dos filas nuevas, Quillay Servicios y Lenga Transportes.

```
sql
%%sql
-- Celda 1.1
INSERT INTO contribuyentes_lab14 VALUES
  ('79412337-8', 'Quillay Servicios SpA', 'PEQUENA'),
  ('77129445-1', 'Lenga Transportes SpA', 'MICRO')
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Entraron las dos filas, con su propia página.

Celda 1.2 · Una reclasificación

La pregunta. ¿Qué deja en la libreta una corrección?

Por qué ahora. Es el segundo movimiento, y es el que después aparece de una forma que sorprende.

En el almacén. Es tachar el renglón de Araucaria y escribirlo de nuevo con el segmento corregido.

La sentencia, parte por parte.

- `UPDATE contribuyentes_lab14` corrige filas que ya existen.
- `SET segmento = 'MEDIANA'` deja el segmento en MEDIANA.
- `WHERE rut = '77884562-8'` apunta solo a Araucaria.

```
sql
%%sql
-- Celda 1.2
UPDATE contribuyentes_lab14
SET segmento = 'MEDIANA'
WHERE rut = '77884562-8'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. La corrección entró, con su página.

Celda 1.3 · Una baja

La pregunta. ¿Qué deja en la libreta un borrado?

Por qué ahora. Es el último movimiento del día.

En el almacén. Es tachar el renglón de Copihue.

La sentencia, parte por parte.

- `DELETE FROM contribuyentes_lab14` borra filas.
- `WHERE rut = '78800840-6'` apunta solo a Copihue.

```
sql
%%sql
-- Celda 1.3
DELETE FROM contribuyentes_lab14 WHERE rut = '78800840-6'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. El borrado entró, con su página.

Celda 1.4 · Las páginas del día

La pregunta. ¿Qué páginas tiene la libreta después del día?

Por qué ahora. De esta lista salen los dos números que vas a escribir en el paso 3.

En el almacén. Es mirar la lista de páginas de la libreta.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo en ella.
- `FROM mi_espacio.contribuyentes_lab14.snapshots` es la vista de sistema con las páginas. Se nombra con tres partes, el espacio, la tabla y la vista, aunque ya hayas hecho `USE`.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 1.4
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.contribuyentes_lab14.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
2909951952428611620  2026-09-25 18:24:57.549  append
6810304009964451568  2026-09-25 18:24:57.980  append
6660392441357658505  2026-09-25 18:25:01.066  overwrite
7477665559979367789  2026-09-25 18:25:02.147  overwrite
4 filas.
```

Cómo se lee. Cuatro páginas, la carga inicial y los tres movimientos del día.

- `2909951952428611620`, a las 18:24:57.549 UTC, es la carga de los cuatro contribuyentes. `append` significa que se agregaron filas.
- `6810304009964451568`, a las 18:24:57.980 UTC, es el `INSERT` de Quillay y Lenga, también `append`. Esta es la **segunda** página, y en este laboratorio hace de marca del proceso nocturno. Todo hasta aquí ya lo copió anoche.
- `6660392441357658505`, a las 18:25:01.066 UTC, es la reclasificación de Araucaria. `overwrite` significa que se reemplazaron filas.
- `7477665559979367789`, a las 18:25:02.147 UTC, las 15:25 en Chile, es la baja de Copihue, también `overwrite`. Es la **última** página.

La baja sale como `overwrite` y no como `delete` porque Copihue compartía papelito con otras filas. Para sacarla, Iceberg reescribió ese papelito sin ella.

Paso 2 · La vista de cambios

Celda 2.1 · Crear la vista

La pregunta. ¿Cómo se le pide a la tabla el movimiento en vez de la foto?

Por qué ahora. Es la herramienta del laboratorio. Primero sobre toda la historia, para ver cómo es.

En el almacén. Es pedirle al almacenero la lista de todo lo que se anotó y se tachó, página por página, desde que abrió la libreta.

La sentencia, parte por parte.

- `CALL spark_catalog.system.create_changelog_view(...)` llama al procedimiento de Iceberg que arma la vista de cambios. `spark_catalog` es el catálogo y `system` el grupo de procedimientos de Iceberg.
- `table => 'mi_espacio.contribuyentes_lab14'` es la tabla, con su espacio, entre comillas. La flecha `=>` pone nombre al argumento.
- Sin más argumentos, la vista cubre toda la historia y se llama como la tabla con `_changes` al final.

```
sql
%%sql
-- Celda 2.1
CALL spark_catalog.system.create_changelog_view(table => 'mi_espacio.contribuyentes_lab14')
```

Lo que sale en pantalla.

```
text
changelog_view
`contribuyentes_lab14_changes`
1 fila.
```

Cómo se lee. Se creó la vista `contribuyentes_lab14_changes`. Las comillas invertidas alrededor del nombre son la forma en que Spark escribe un nombre de objeto.

La vista vive en tu sesión, no en el catálogo. No aparece en `SHOW TABLES`, y si reinicias el kernel desaparece. No copia datos, solo guarda cómo calcular los cambios cuando la consultes.

Celda 2.2 · Leer toda la historia

La pregunta. ¿Qué movimientos tuvo la tabla desde que nació?

Por qué ahora. Para ver cómo devuelve Iceberg cada tipo de cambio.

En el almacén. Es leer la lista de anotaciones y tachones, en orden de página.

La sentencia, parte por parte.

- `SELECT *` trae todas las columnas de la vista, las de la tabla más tres nuevas.
- `FROM contribuyentes_lab14_changes` es la vista del paso anterior. No lleva el espacio adelante porque vive en tu sesión.
- `ORDER BY _change_ordinal, rut` ordena por página y, dentro de cada página, por RUT.

```
sql
%%sql
-- Celda 2.2
SELECT * FROM contribuyentes_lab14_changes
ORDER BY _change_ordinal, rut
```

Lo que sale en pantalla.

```
text
rut          razon_social          segmento _change_type _change_ordinal
_commit_snapshot_id
76171162-8  Huemul Alimentos EIRL          GRANDE  INSERT      0
2909951952428611620
77746521-K  Pehuen Logistica EIRL          MICRO  INSERT      0
2909951952428611620
77884562-8  Araucaria Ferreteria SpA       PEQUENA INSERT      0
2909951952428611620
78800840-6  Copihue Maquinarias EIRL       MEDIANA INSERT      0
2909951952428611620
77129445-1  Lenga Transportes SpA          MICRO  INSERT      1
6810304009964451568
79412337-8  Quillay Servicios SpA          PEQUENA INSERT      1
6810304009964451568
77884562-8  Araucaria Ferreteria SpA       MEDIANA INSERT      2
6660392441357658505
77884562-8  Araucaria Ferreteria SpA       PEQUENA DELETE      2
6660392441357658505
78800840-6  Copihue Maquinarias EIRL       MEDIANA DELETE      3
7477665559979367789
9 filas.
```

Cómo se lee. Nueve movimientos repartidos en las cuatro páginas.

Las tres columnas nuevas son las que importan.

- `_change_type` dice qué pasó con la fila, `INSERT` si entró o `DELETE` si salió.
- `_change_ordinal` numera las páginas desde cero, en orden, para poder ordenar.
- `_commit_snapshot_id` es el número de la página en que ocurrió, el mismo de la celda 1.4.

Ahora página por página.

- Ordinal 0, página `2909951952428611620`. Los cuatro contribuyentes de la carga inicial, cada uno como `INSERT`.
- Ordinal 1, página `6810304009964451568`. Lenga y Quillay, los dos nuevos, como `INSERT`.
- Ordinal 2, página `6660392441357658505`. Araucaria aparece dos veces, un `DELETE` con PEQUENA y un `INSERT` con MEDIANA. Un `UPDATE` no aparece como una modificación. Aparece como la salida de la fila vieja y la entrada de la nueva, en la misma página.
- Ordinal 3, página `7477665559979367789`. Copihue como `DELETE`, la baja.

Paso 3 · Solo lo de hoy

Celda 3.1 · La vista desde la marca

La pregunta. ¿Cómo se piden solo los cambios desde la última vez que corrió el proceso?

Por qué ahora. Un proceso nocturno no quiere toda la historia. Quiere lo que pasó desde su marca.

En el almacén. Es decirle al almacenero desde qué página ya copiaste y hasta cuál quieres, y que te dé solo lo que hay entre medio.



Página 1 (gris). Ya está en el destino desde antes.

Página 2 · start (azul). Es la marca del proceso, la que va en `start-snapshot-id`. No se incluye, porque ya se copió.

Página 3 y Página 4 · end (verde). Lo que falta. La 4 va en `end-snapshot-id` y sí se incluye.

desde_ayer (amarillo). La vista nueva, con solo los cambios de las páginas 3 y 4.

Aquí escribes dos números que salen de tu pantalla. En `<TU_SEGUNDO_SNAPSHOT_ID>` va el `snapshot_id` de la segunda fila de la celda 1.4, y en `<TU_ULTIMO_SNAPSHOT_ID>` el de la última fila. En la solución fueron `6810304009964451568` y `7477665559979367789`. Los tuyos son otros, porque cada tabla numera sus páginas a su manera. Van entre comillas simples, como texto, porque las opciones de `map` son todas texto.

La sentencia, parte por parte.

- `CALL spark_catalog.system.create_changelog_view(...)` es el mismo procedimiento del paso 2.
- `table => 'mi_espacio.contribuyentes_lab14'` es la tabla.
- `options => map('start-snapshot-id', '<TU_SEGUNDO_SNAPSHOT_ID>', 'end-snapshot-id', '<TU_ULTIMO_SNAPSHOT_ID>')` son las opciones en pares nombre y valor. `map` arma esos pares. `start-snapshot-id` es la página desde la cual mirar, sin incluirla, y `end-snapshot-id` la página hasta donde mirar, incluida.
- `changelog_view => 'desde_ayer'` le pone otro nombre a la vista, para no pisar la del paso 2.

```
sql
%%sql
-- Celda 3.1
CALL spark_catalog.system.create_changelog_view(
  table      => 'mi_espacio.contribuyentes_lab14',
  options    => map('start-snapshot-id', '<TU_SEGUNDO_SNAPSHOT_ID>',
                  'end-snapshot-id',   '<TU_ULTIMO_SNAPSHOT_ID>'),
  changelog_view => 'desde_ayer'
)
```

Lo que sale en pantalla.

```
text
changelog_view
desde_ayer
1 fila.
```

Cómo se lee. Se creó la vista `desde_ayer`.

Celda 3.2 · Leer lo de hoy

La pregunta. ¿Qué cambió desde la marca?

Por qué ahora. Es lo que el proceso nocturno tiene que aplicar en el destino.

En el almacén. Es leer solo las anotaciones de las páginas que faltaban.

La sentencia, parte por parte.

- `SELECT *` trae todas las columnas de la vista.
- `FROM desde_ayer` es la vista de la celda 3.1.
- `ORDER BY _change_ordinal, rut` ordena por página y por RUT.

```
sql
%%sql
-- Celda 3.2
SELECT * FROM desde_ayer
ORDER BY _change_ordinal, rut
```

Lo que sale en pantalla.

```
text
rut          razon_social          segmento _change_type _change_ordinal
_commit_snapshot_id
77884562-8  Araucaria Ferreteria SpA  PEQUENA  DELETE      0
6660392441357658505
77884562-8  Araucaria Ferreteria SpA  MEDIANA  INSERT      0
6660392441357658505
78800840-6  Copihue Maquinarias EIRL  MEDIANA  DELETE      1
7477665559979367789
3 filas.
```

Cómo se lee. Tres filas, la reclasificación de Araucaria y la baja de Copihue.

- Araucaria sale dos veces en la página `6660392441357658505`, el `DELETE` con `PEQUENA` y el `INSERT` con `MEDIANA`.
- Copihue sale como `DELETE` en la página `7477665559979367789`.

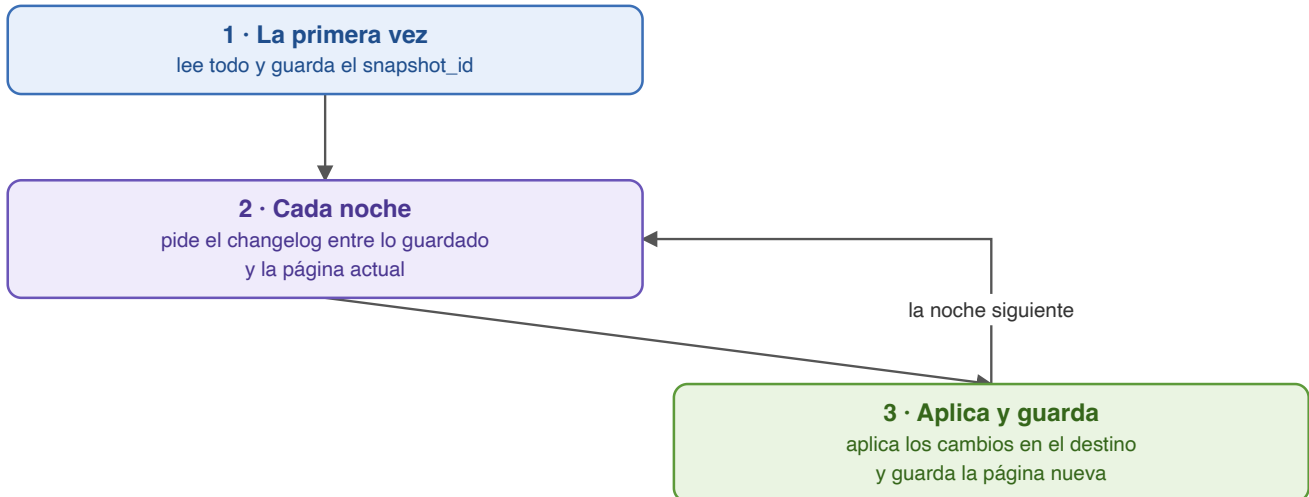
Lenga y Quillay no están, porque entraron en la página que pusiste como punto de partida, y esa no se incluye. En esta vista `_change_ordinal` vuelve a partir de cero, porque cuenta las páginas del rango pedido y no las de toda la tabla.

Paso 4 · Cómo lo usa un proceso nocturno

Este paso no tiene celdas. Es texto en el cuaderno, y aquí va explicado.

La pregunta. ¿Cómo se arma un proceso incremental con esto?

En el almacén. El contador anota en su cuaderno hasta qué página copió. Cada noche pide lo que hay desde esa página hasta la última, lo copia y anota la nueva página. Si se le corta la luz a mitad, vuelve a pedir lo mismo y le dan lo mismo.



1 · La primera vez (azul). El proceso lee la tabla entera y guarda el `snapshot_id` que procesó, en un archivo o en una tabla de control.

2 · Cada noche (morado). Pregunta cuál es la página actual y pide el changelog entre la que guardó y esa.

3 · Aplica y guarda (verde). Aplica solo esos cambios en el destino y, si salió bien, guarda la página nueva como su marca.

Tres propiedades hacen que esto funcione en producción.

- **No relee nada.** El costo de la carga nocturna depende de lo que cambió, no del tamaño de la tabla.
- **Es repetible.** Si el proceso se cae a mitad, vuelve a pedir el mismo rango y obtiene lo mismo.
- **No hace falta tocar el origen.** No hay columna de auditoría que mantener, ni disparadores, ni bitácora al lado.

El rango deja de existir si alguien bota esas páginas con `expire_snapshots`. La retención de páginas tiene que ser más larga que el intervalo del proceso que consume. Si el proceso corre cada noche y se botan las páginas a diario, un fin de semana largo lo deja sin rango.

Preguntas frecuentes

¿Por qué un UPDATE sale como DELETE e INSERT?

Porque así guarda Iceberg una corrección. La fila vieja deja de estar y entra la nueva, en la misma página. Si el destino necesita saber que fue una corrección y no una baja más un alta, puede juntar las dos filas por su llave, el RUT, dentro de la misma página.

¿Por qué start-snapshot-id no se incluye?

Porque es la marca de lo que ya se copió. Si se incluyera, el proceso copiaría dos veces los cambios de esa página. Por eso se pasa la última página procesada, y el rango parte en la siguiente.

¿Dónde guardo la marca en un proceso real?

En cualquier lugar que sobreviva al proceso. Una tabla de control con una fila por tabla consumida, con el nombre de la tabla y el último `snapshot_id` aplicado, es lo más común. Se actualiza solo después de aplicar los cambios con éxito.

¿Por qué la vista desaparece si reinicio el kernel?

Porque vive en la sesión de Spark, no en el catálogo. Es una vista temporal. Se vuelve a crear con la misma sentencia, y como las páginas siguen en la libreta, devuelve lo mismo.

¿Qué pasa si alguien botó las páginas que necesito?

El procedimiento falla, porque ya no puede leer lo que pasó en ellas. El proceso tiene que volver a la primera vez, leer la tabla entera y guardar una marca nueva. Por eso la retención de páginas se decide mirando cada cuánto corren los procesos que consumen la tabla.

¿Por qué la baja de Copihue es overwrite y no delete?

Porque Copihue compartía papelito con otras filas. Para sacarla, Iceberg escribió un papelito nuevo sin ella y dejó de usar el anterior, y eso es un `overwrite`. Si hubiera estado sola en su papelito, bastaba con dejar de nombrarlo y la página habría sido `delete`. Para la vista de cambios da lo mismo, igual sale como un `DELETE`.

¿Por qué las horas no calzan con mi reloj?

Porque Spark muestra las horas en UTC, la hora universal. Chile en septiembre está tres horas atrás, así que las 18:25 UTC son las 15:25 en Chile.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

Dos formas de preguntarle a la libreta, y no se parecen. Una devuelve cómo estaba la tabla en una página. La otra devuelve qué pasó entre dos páginas.

En el almacén. El contador ya no copia la libreta entera cada noche. Pide solo lo que se anotó y se tachó desde la última página que copió, y eso ya estaba escrito en la libreta desde el principio.

Lo que pides	Lo que te devuelve	Para qué sirve
VERSION AS OF	la foto completa en una página	auditar, comparar, revertir
create_changelog_view	el movimiento entre dos páginas	alimentar a otro sistema

Los números de la solución ejecutada.

Vista	Páginas que cubre	Movimientos
contribuyentes_lab14_changes , celda 2.2	las cuatro	9 filas
desde_ayer , celda 3.2	la tercera y la cuarta	3 filas

Con esto, ninguna herramienta necesita una copia propia de la tabla ni un proceso que la recorra entera cada noche. Piden lo que cambió, y lo que cambió estaba escrito desde el principio.